Structured Programming Languages are using high level syntaxes, which are available in more number and which may provide more no of features to the applications.

4) Unstructured Programming Languages are using only "goto" statement to define flow of execution, which is not sufficient to provide very good flow of execution in applications.

   Structured Programming Languages are using more and more no of flow controllers like if, switch, for, while, do-while, break,..... to defined very good flow of execution in applications.

5) Unstructured Programming languages are not having functions feature, it will increase code redundancy [Code duplication], it is not suggestible in application development.

   Structured Programming languages are having functions feature to improve code reusability.

Q) What are the differences between Structured Programming Languages and Object Oriented Programming Languages?

1) Structured Programming Languages are providing difficult approach to prepare applications.
   EX: C, PASCAL,...

   Object Oriented Programming Languages are providing very simplified approach to prepare applications.
   EX: JAVA, C++,....

2) Structured Programming languages are not having modularity.
   Object Oriented Programming Languages are having very good modularity.

3) Structured Programming languages are not having very good abstraction.
   Object Oriented Programming Languages are having very good abstraction levels.

4) Structured Programming languages are not providing very good security for the applications.
   Object oriented programming Languages is providing very good security for the application data.

5) Structured Programming languages are not providing very good sharability.
   Object oriented programming Languages are providing very good sharability.

6) Structured Programming Languages are not providing very good code reusability.

Object oriented programming languages are providing very good code reusability.

Aspect Oriented Programming Languages:

If we want to prepare applications by using Object orientation then we have to provide both business logic and Services logic in combined manner, it will provide tightly coupled design, it will reduce sharability and code reusability.

To overcome the above problem we have to use Aspect Orientation. Aspect Orientation is a methodology or a set of rules and regulations or a set of guide lines which are applied on object oriented programming to get loosely coupled design in order to improve sharability and code reusability.

In case of Aspect orientation, we will separate all services from business logic, we will declare each and every service as an aspect and we will inject these services to the applications at runtime as per the requirement.

Object Oriented Features:

To show the nature of Object orientation, Object Orientation has provided the following features.

1) **Class**
2) **Object**
3) **Encapsulation**
4) **Abstraction**
5) **Inheritance**
6) **Polymorphism**
7) **Message Passing**

There are two types of programming languages on the basis of object oriented features.

1) **Object Oriented Programming Languages**
2) **Object based Programming Languages**

Q) What is the difference between Object Oriented Programming Languages and Object Based programming languages?

- **Object Oriented Programming languages are able to allow all the object oriented features including "Inheritance".**
  EX: Java

- **Object based programming languages are able to allow all the object oriented features excluding "Inheritance".**
  EX: Java Script

Q) What are the differences between class and Object?

1) **Class is a group of elements having common properties and behaviors.**
   Object is an individual element among the group of elements having physical behaviors and physical properties

2) **Class is virtual.**
   Object is real

3) **Class is virtual encapsulation of properties and behaviors.**
   Object is physical encapsulation of properties and behaviors.

4) **Class is Generalization.**
   Object is Specialization.

5) **Class is Model or Blue Print for the objects.**
   Object is an instance of the class.

Q) What is the difference between Encapsulation and Abstraction?

The process of binding data and coding part is called as "Encapsulation".

The process showing necessary data or implementation and hiding unnecessary data or implementation is called as "Abstraction".

Note: In Object Oriented programming languages, both encapsulation and abstraction are improvising "Security".

5.Inheritance:

The process of getting variables and methods from one class to another class is called as "Inheritance".

The main objective of inheritance is "Code Reusability".

EX:

```
1)  class Employee {
2)        String eid;
3)        String ename;
4)        String eaddr;
5)        ---
6)        ---
7)        void getEmpDetails() {
8)             ----
9)        }
10) }
11) class Manager extends Employee {
12)       --- Reuse variables and methods of Employee class---
13) }
14) class Accountant extends Employee {
15)        --- Reuse variables and methods of Employee class here----
16) }
17) class Engineer extends Employee {
18)       --- Reuse variables and methods of Employee class here----
19) }
```

6.Polymorphism:

- Polymorphism is "Greak" word, where poly means many and morphism means structures [forms].
- If one thing is existed in multiple forms then it is called as Polymorphism.
- The main advantage of Polymorphism is "Flexibility" to design application.

7.Message Passing:

The process of transferring data along with flow of execution from one instruction to another instructions is called as Message passing.

The main advantage of message passing is to improve "Communication" between entities and "data navigation" between entities.

Containers in Java:

Container is a Java element, it able to manage some other programming elements like variables, methods, blocks,.....

There are three types of containers in JAVA.

1) Class
2) Abstract Class
3) Interface

1) Class:

The main Purpose of the class is to represent all real world entities in Java applications.
EX: Student, Employee, Product, Account,….

To represent entities data in Java applications we will use Variables.
To represent entities behaviours we will use methods.

Syntax:

```
[Access_Modifiers] class Class_Name [extends Super_Class_Name]
        [implements interface_List]
        {
                --- variables-----
                -- methods--------
                ----constructors-----
                ----blocks------
                ---classes-------
                ----abstract classes-----
                ---interfaces------
                ---- enums--------
        }
```

Access Modifiers:

There are two types of Access modifiers

1.To define scopes to the programming elements, there are four types of access modifiers. public, protected, <default> and private.
Where public and <default> are allowed for classes, protected and private are not allowed for classes.

Note: All public, protected , <default> and private are allowed for inner classes.
Note:Where private and protected access modifiers are defined on the basis of classes boudaries, so that,they are not applicable for classes, they are applicable for members of the classes including inner classes.

2.To define some extra nature to the programming elements , we have the following access modifiers.

static, final, abstract, native, volatile, transient, synchronized, strictfp,.....

Where final, abstract and strictfp are allowed for the classes.

Note: The access modifiers like static, abstract, final and strictfp are allowed for inner classes.

Note: Where static keyword was defined on the basis of classes origin, so that, it is not applicable for classes, it is applicable for members of the classes including inner classes.

Where 'class' is a java keyword, it can be used to represent 'Class' object oriented feature.

Where 'Class_Name' is name is an identifier, it can be used to recognize the classes individually.

Where 'extends' keyword is able to specify a particular super class name in class syntax in order to get variables and methods from the specified super class to the present java class.

Note: In class syntax, 'extends' keyword is able to allow only one super class name, it will not allow more than one super class, because, it will represent multiple inheritance, it is not possible in Java.

Where 'implements' keyword is able to specify one or more no of interfaces in class syntax in order to provide implementation for all abstract methods of that interfaces in the present class.

Note: In class syntax, extends keyword is able to allow only one super class, but, implements keyword is able to allow more than one interface.

Note: In class syntax, 'extends' and 'implements' keywords are optional, we can write a class with extends keyword and without implements keyword, we can write a class with implements keyword and without extends keyword, we can write a class without both extends and implements keywords, if we want to write both extends and implements keywords first we have to write extends keyword then only we have to write implements keyword, we must not interchange extends and implements keywords.

1) **class A{ } ----> valid**
2) **public class A{ }----> valid**
3) **private class A{ }----> Invalid**
4) **protected class A{ }----> Invalid**

5) class A{ public class B{ } }----> valid
6) class A{ private class B{ } }----> valid
7) class A{ protected class B { } }----> valid
8) class A{ class B{ } }----> valid
9) static class A{ }----> Invalid
10) final class A{ }----> valid
11) abstract class A{ }----> valid
12) synchronized class A{ }----> Invalid
13) native class A{ }----> Invalid
14) class A{ static class B{ } }----> valid
15) class A{ abstract class B{ } }----> valid
16) class A{ strictfp class B{ } }----> valid
17) class A{ native class B{ } }----> Invalid
18) class A extends B{ }----> valid
19) class A extends A{ }----> Invalid, cyclic inheritance Error
20) class A extends B, C{ }----> Invalid
21) class A implements I{ }----> valid
22) class A implements I1, I2{ }----> valid
23) class A implements I extends B{ }----> Invalid
24) class A extends B implements I{ }----> valid
25) class A extends B implements I1, I2{ }----> valid

Procedure To Write Classes In Java Applications:

1) Declare a class by using 'class' keyword.
2) Declare variables and methods in class as per the requirement.
3) In main class and in main() method, create object for the respective class.
4) Access members of the class by using the generated reference variable.

Entities[Student, Customer, Employee,...]-----> classes
Entities data[sid, sname, saddr,...] ----> variables
Entities actions or behaviours[add, search, delete,..] ----> methods

EX:

```
1)  class Employee
2)  {
3)     String eid="E-111";
4)     String ename="Durga";
5)     float esal=25000.0f;
6)     String eaddr="Hyderabad";
7)     String eemail="durga@durgasoft.com";
8)  String emobile="91-9988776655";
9)     public void display_Emp_Details()  {
10)       System.out.println("Employee Details");
11)       System.out.println("-------------------");
```

```
12)       System.out.println("Employee Id    :"+eid);
13)       System.out.println("Employee Name  :"+ename);
14)       System.out.println("Employee Saslary:"+esal);
15)       System.out.println("Employee Address:"+eaddr);
16)       System.out.println("Employee Email  :"+eemail);
17)       System.out.println("Employee Mobile :"+emobile);
18)    }
19) }
20) class Test
21) {
22)    public static void main(String[] args)
23)    {
24)       Employee emp = new Employee();
25)       emp.display_Emp_Details();
26)    }
27) }
```

There are two types of methods.

1) Concrete methods
2) Abstract methods

Q) What are the differences between *Concrete methods* and *abstract methods*?

1) Concrete method is a method, it will have both method declaration and method implementation.

   EX: void add(int i, int j)//Method Declaration
   {// Method implementation started here
   int k=i+j;
   System.out.println(k);
   }// Method Implementation ended here

   Abstract method is a method, it will have only method declaration.

   EX: abstract void add(int i, int j);

2) To declare concrete methods, no need to use any special keyword.
   To declare abstract method, we must use abstract keyword.
3) Concrete methods are allowed in classes and in abstract classes.
   Abstract methods are allowed in abstract classes and interfaces.
4) Concrete methods will provide less sharability.
   Abstract methods will provide more sharability.

2) Abstract Classes:

Abstract class is a java class, it able to allow zero or more no of concrete methods and zero or more no of abstract methods.

Note: To declare abstract classes, it is not at all mandatory condition to have at least one abstract method, we can declare abstract classes with 0 no of abstract methods, but, if we want to declare a method as an abstract method then the respective class must be abstract class.

For abstract classes, we are able to create only reference variables; we are unable to create objects.

```
abstract class A {
      ----
}
A  a  =  new  A();--> Error
A a = null; → No Error
```

Abstract classes are able to provide more sharability when compared with classes.

Procedure to use Abstract Classes:

1) Declare an abstract class with "abstract".
2) Declare concrete methods and abstract methods in abstract class as per the requirement.
3) Declare sub class for abstract class.
4) Implement abstract methods in sub class.
5) In main class and in main() method, create object for sub class and declare reference variable either for abstract class or for sub class.
6) Access abstract class members.

Note: If we declare reference variable for abstract class then we are able to access only abstract class members, but, if we declare reference variable for sub class then we are able to access both abstract class members and sub class members.

EX:

```
1)  abstract class A
2)  {
3)     void m1()
4)     {
5)        System.out.println("m1-A");
6)     }
7)     abstract void m2();
8)     abstract void m3();
9)  }
10) class B extends A
```

```
11) {
12)     void m2()
13)     {
14)         System.out.println("m2-B");
15)     }
16)     void m3()
17)     {
18)         System.out.println("m3-B");
19)     }
20)     void m4()
21)     {
22)         System.out.println("m4-B");
23)     }
24) }
25) class Test
26) {
27)     public static void main(String[] args)
28)     {
29)         //A a=new A();--> Error
30)         A a = new B();
31)         a.m1();
32)         a.m2();
33)         a.m3();
34)         //a.m4();--> Error
35)         B b = new B();
36)         b.m1();
37)         b.m2();
38)         b.m3();
39)         b.m4();
40)     }
41) }
```

Q) What are the differences between *Concrete Classes* and *Abstract Classes?*

1) Classes are able to allow only concrete methods.
   Abstract classes are able to allow both concrete methods and abstract methods.

2) To declare concrete classes, only, 'class' keyword is sufficient.
   To declare abstract classes we need to use 'abstract' keyword along with class keyword.

3) For classes, we are able to create both reference variables and objects,
   For abstract classes, we are able to create only reference variables; we are unable to create objects.

4) Concrete classes will provide less sharability.
Abstract classes will provide more sharability.

3) Interfaces:

- Interface is a java feature, it able to allow zero or more no of abstract methods only.
- For interfaces, we are able to declare only reference variables; we are unable to create objects.
- In case of interfaces, by default, all the variables as "public static final".
- In case of interfaces, by default, all the methods are "public and abstract".
- When compared with classes and abstract classes, interfaces will provide more sharability.

Procedure to Use interfaces in Java applications:

1) Declare an interface with "interface" keyword.
2) Declare variables and methods in interface as per the requirement.
3) Declare an implementation class for interface by including "implements" keyword.
4) Provide implementation for abstract methods in implementation class which are declared in interface.
5) In main class, in main() method, create object for implementation class ,but, declare reference variable either for interface or for implementation class.
6) Access interface members.

Note: If declare reference variable for interface then we are able to access only interface members, we are unable to access implementation class own members. If we declare reference variable for implementation class then we are able to access both interface members and implementation class own members.

Example:

```
1)  interface I
2)  {
3)     int x=20;// public static final
4)     void m1();// public and abstract
5)     void m2();// public and abstract
6)     void m3();// public and abstract
7)  }
8)  class A implements I
9)  {
10)    public void m1()
```

```
11)   {
12)      System.out.println("m1-A");
13)   }
14)   public void m2()
15)   {
16)      System.out.println("m2-A");
17)   }
18)   public void m3()
19)   {
20)      System.out.println("m3-A");
21)   }
22)   public void m4()
23)   {
24)      System.out.println("m4-A");
25)   }
26) }
27) class Test
28) {
29)   public static void main(String[] args)
30)   {
31)      //I i=new I();---> Error
32)      I i=new A();
33)      System.out.println(I.x);
34)      System.out.println(i.x);
35)      i.m1();
36)      i.m2();
37)      i.m3();
38)      //i.m4();----> Error
39)      A a=new A();
40)      System.out.println(a.x);
41)      System.out.println(A.x);
42)      a.m1();
43)      a.m2();
44)      a.m3();
45)      a.m4();
46)   }
47) }
```

Q) What are the differences between Class, abstract Clacss and Interface?

1) Class is able to allow concrete methods only.
   Abstract class is able to allow both concrete methods and abstract methods
   Interface is able to allow abstract methods only.

2) To declare class, only, class keyword is sufficient.

To declare abstract class, we have to use "abstract" keyword along with class keyword.
To declare interface we have to use "interface" keyword explicitly.

3) For classes only, we are able to create both reference variables and objects.
For abstract classes and interfaces, we are able to declare reference variables; we are unable to create objects.
4) In case of interfaces, by default, all variables are "public static final".
In case of classes and abstract classes, no default cases for variables.

5) In case of interfaces, by default, all methods are "public and abstract".
In case of classes and abstract classes, no default cases for methods.

6) Constructors are allowed in classes and abstract classes.
Constructors are not allowed in interfaces.

7) Classes are able to provide less sharability.
Abstract classes are able to provide moddle level sharability.
Interfaces are able to provide more sharability.

Methods In Java:

Method is a set of instructions, it will represent a particular action in java applications.

Syntax:

```
[Access_Modifiers] return_Type method_Name([param_List])[throws Exception_List]
{
  ----- instructions to represent a particular action-----
}
```

Java methods are able to allow the access modifiers like public, protected, <default> and private.
Java methods are able to allow the access modifiers like static, final, abstract, native, synchronized, strictfp.

Where the purpose of return_Type is to specify which type of data the present method is returning. In java applications, all primitive data types, all user defined data types and 'void' are allowed for methods as return types.

Note: 'void' return type is representing "Nothing is returned" from methods.

Where "method_name" is an identifier, it can be used to recognize the methods individually.

Where param_List can be used to pass some input data to the methods in order to perform an action. In java applications, we are able to provide all primitive data types and all user defined data types as parameter types.

Where "throws" keyword can be used to bypass or delegate the generated exception from present method to caller method of the present method

To describe method information, there are two approaches.

1) **Method Signature**
2) **Method Prototype**

Q) What is the difference between *Method Signature* and *Method Prototype?*

Method signature is the description of the method, it includes method name and parameter list.
EX: forName(String Class_Name)

Method prototype is the description of the method, it will include access modifiers list, return type, method name, parameters list, throws exception list.
EX:public static Class forName(String class_Name)throws ClassNotFoundException

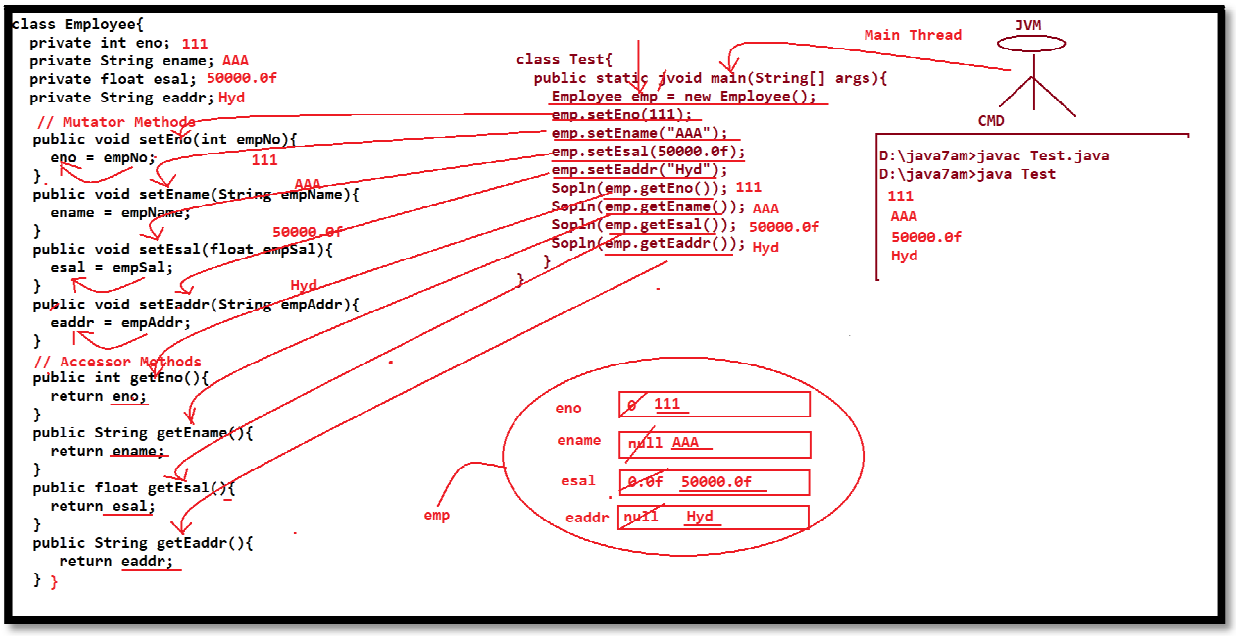
There are two types of methods in Java w.r.t the Object state manipulations.

1) **Mutator Methods**
2) **Accessor Methods**

Q) What are the differences between *Mutator methods* and *Accessor methods?*

Mutator methods are the java methods, which are used to set/modify data in Objects.
EX: All setXXX(--) methods in Java Bean classes are mutator methods.

Accessor methods are the java methods, which are used to get/access data from Objects.
EX:All getXXX() methods in java bean classes are Accessor methods



Note: Java bean is a normal java class, it will include properties and the respective setXXX(-) methods and getXXX() methods.

Variable-Argument Method [Var-Arg method]

In general, in java applications, if we declare any method with 'n' no of parameters then we must access that method with the same 'n' no of parameter values , it is not possible to access that method by passing 'n+1' no of parameter values and 'n-1' no of parameter values.

EX:

```
1)  class A
2)  {
3)     void add(int i, int j)
4)     {
5)        System.out.println(i+j);
6)     }
7)  }
8)  class Test
9)  {
10)    public static void main(String[] args)
11)    {
12)       A a=new A();
13)       a.add(10,20);// Valid.
14)       //a.add();--> Invalid
15)       //a.add(10);--> Invalid
16)       //a.add(10,20,30);--> Invalid
17)    }
```

```
18) }
```

As per the requirement, we want to access a method with variable no of parameter values [0 no of params or 1 no of params or 2 no of params,....]

To achieve this requirement we have to use "Var-Arg Method" provided by JDK5.0 version.

Var-Arg method is a java method including Var-Arg parameter.
Syntax for Var-Arg Parameter:

Data_Type ... Var_Name

EX: for var-Arg method:

```
void m1(int ... a)//int[] a={---- argumentvalues-----}
{
  -----
}
```

When we access Var-Arg method with 'n' no of parameter values then all these 'n' no of parameter values are stored in the form of Array which is generated from var-ARg parameter.

EX:

```
1)  class A
2)  {
3)     void add(int ... a)// int[] a={--- listof arguments---}
4)     {
5)        System.out.println("No Of Arguments  :"+a.length);
6)        int result=0;
7)        System.out.print("Argument List    :");
8)        for(int i=0;i<a.length;i++)
9)        {
10)          System.out.print(a[i]+"  ");
11)          result=result+a[i];
12)       }
13)       System.out.println();
14)       System.out.println("Addition        :"+result);
15)       System.out.println("--------------------------");
16)    }
```

```
17) }
18) class Test
19) {
20)     public static void main(String[] args)
21)     {
22)         A a=new A();
23)         a.add();
24)         a.add(10);
25)         a.add(10,20);
26)         a.add(10,20,30);
27)     }
28) }
```

Note: In Var-Arg methods, normal parameters are allowed along with Var-Arg parameter but they must be provided before var-Arg parameter, not after var-Arg parameter, because, var-Arg parameter must be last parameter in var-Arg method.

Note: Due to the above reason, Var-Arg methods are able to allow at most one Var-Arg parameter, not allowing more than one Var-Arg parameters.

EX:

1) void m1(int ... i, float f){ } ----> Invalid
2) void m1(float f, int ... i){ } ----> valid
3) void m1(int ... i, float ... f){ } --->Invalid

Object Creation Process:

Q)What is the Requirement to Create Objects in Java?

1) To store entities data temporarily in java applications we need objects.
2) To Access the members of any particular class we need objects.

If we want to create Objects we have to use the following syntax.
Class_Name ref_Var = new Class_Name([param_List]);

EX:

```
class A{
----
}

A  a = new  A();
```

When JVM encounter the above instruction, JVM will perform the following actions.

1) Creating memory for the Object
2) Generating Identities for the object

3) Providing initializations inside the Object

1) Creating Memory for the object:

When JVM encounter "new" keyword in Object creation statement then JVM will check to which class we are creating object on the basis of Constructor name then JVM will load the respective class byte code to the memory [Method Area]

After loading class byte code, JVM will identify minimal object memory size by recognizing all the instance variables and their data types

After getting memory size, JVM will send a request to Heap manager about to create an object with the specified minimal memory.

As per JVM requirement, Heap Manager will create the required block of memory at Heap Memory.

2) Generating Identities for the Object:

After creating a block of memory [Object] for JVM requirements, Heap manager will assign an integer value as an identity for the object called as "HashCode".

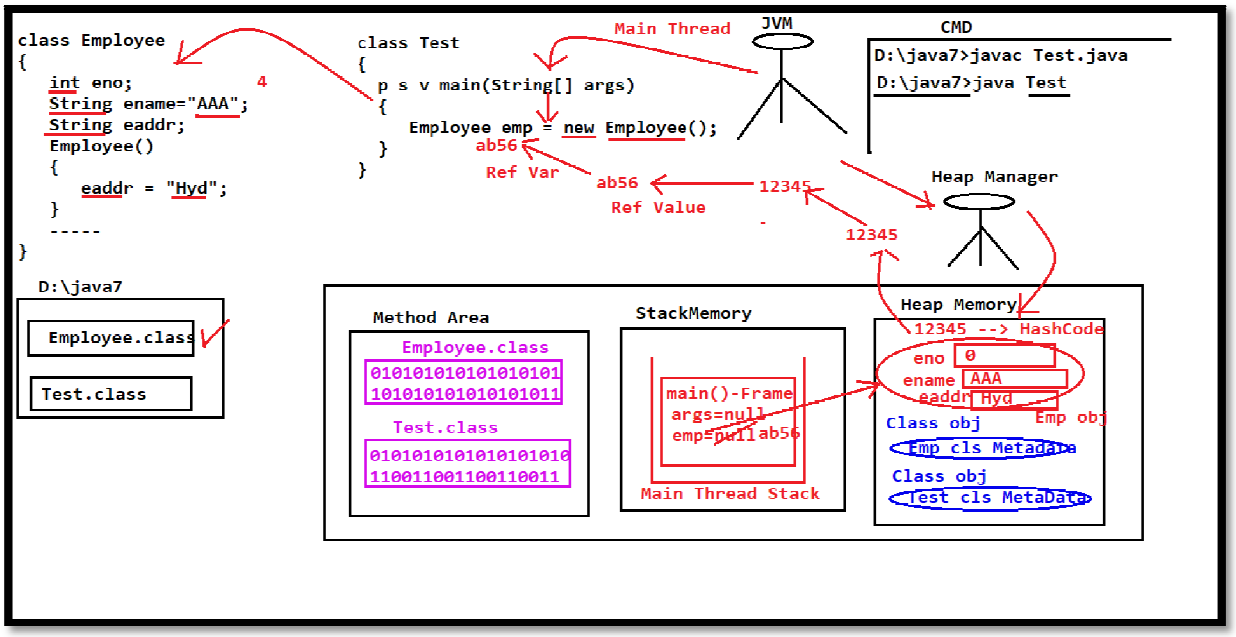
After getting HashCode value, Heap Manager will send that hashCode value to JVM, where JVM will convert that hashCode value to its Hexa Decimal form called as "Reference Value".

After getting Object reference value, JVM will assign that reference value to a variable called as "Reference Variable".

3) Providing initializations inside the Object:

After creating object and its identities, JVM will allocate memory for all the instance variables inside the object on the basis of their data types.

After getting memory for the instance variables, JVM will provide initial values to the instance variables by searching initializations at class level declaration and at constructor.
If any instance variable is not having initialization at both constructor and at class level declaration then JVM will store default value on the basis of their data type as initial value inside the object.



Note: In java applications, when a class byte code is loaded to the memory, automatically, JVM will create java.lang.Class object at heap memory with the metadata of the loaded class.

Note: In java application, when we create a thread [Main Thread or User Thread], automatically, a stack will be created at stack memory and it able to store methods details in the form of Frames which are accessed by the respective thread.

To get hashcode value and reference value of an object we have to use the following two methods.

public native int hashCode()
public String toString()

Note: Native method is a java method, declared in Java, but, implemented in non Java programming languages may be C, ASL, ...

EX:

```
1)  class A
2)  {
3)  }
4)  class Test
5)  {
6)     public static void main(String[] args)
7)     {
8)        A a=new A();
9)        int hashCode=a.hashCode();
```

```
10)     System.out.println("Object HashCode  :"+hashCode);
11)     String ref=a.toString();
12)     System.out.println("Object Reference :"+ref);
13)   }
14) }
```

OUTPUT:
Object Hashcode: 31168322
Object Reference: A@adb9742

In Java applications, there is a common and default super class for each and every java class [predefined classes and user defined classes] that is "java.lang.Object" class, where java.lang.Object class contains the following 11 methods in order to share to all java classes.

1) hashCode()
2) toString()
3) getClass()
4) clone()
5) equals(Object obj)
6) finalize()
7) wait()
8) wait(long time)
9) wait(long time, int time)
10) notify()
11) notifyAll()

Q) If we extend a class to some other class explicitly then the respective sub class is having two super classes [default super class [Object] and explicit super class], it represents multiple inheritance, how can we say multiple inheritance is not possible in JAVA?

In java, java.lang.Object class is common and default super class for every class when that class is not extending from any other super class explicitly. If our class is extending some other class explicitly then Object class is not directly super class to the respective sub class, Object class is indirectly super class to the respective sub class through the explicit super class , that is, Object class is not super class to the respective sub class through Multiple Inheritance, that is through Multi Level Inheritance.

EX:

```
class A{
---
}
class B extends A{
```

}

Note: Here Object class is super class to A, A is super class to B
Object <--- A <--- B

In java applications, when we pass a particular class object reference variable as parameter to System.out.println(-) method then JVM will access toString() over the provided reference variable internally.

EX:

```
1)  class A
2)  {
3)  }
4)  class Test
5)  {
6)     public static void main(String[] args)
7)     {
8)        A a=new A();
9)
10)       String ref=a.toString();
11)       System.out.println(ref);
12)
13)       System.out.println(a.toString());
14)
15)       System.out.println(a);//System.out.println(a.toString());
16)    }
17) }
```

OUTPUT:
A@1db9742
A@1db9742
A@1db9742

In the above program, when we access toString() method internally or externally , first, JVM has to execute toString(), to execute toString() method JVM will search for toString() method in the respective class whose reference variable we passed as parameter to System.out.println(--) method, if the required toString() method is not existed in the respective class then JVM will search for toString() method in its super class, here if super class is not existed then JVM will search for toString() method in the common and default super class Object class. In Object class, toString() method was implemented insuch a way to return a string contains "Class_Name@ref_Val" .

As per the requirement, if we want to display our own data instead of Object reference value when we pass reference variable as parameter to System.out.println(-) method then we have to provide our own toSrtring() method in the respective class.

EX:

```
1)  class Employee
2)  {
3)     String eid="E-111";
4)     String ename="Durga";
5)     float esal=50000.0f;
6)     String eaddr="Hyd";
7)     String eemail="durga@durgasoft.com";
8)     String emobile="91-9988776655";
9)
10)    public String toString()
11)    {
12)       System.out.println("Employee Details");
13)       System.out.println("----------------------");
14)       System.out.println("Employee Id     :"+eid);
15)       System.out.println("Employee Name   :"+ename);
16)       System.out.println("Employee Salary  :"+esal);
17)       System.out.println("Employee Address :"+eaddr);
18)       System.out.println("Employee Email   :"+eemail);
19)       System.out.println("Employee Mobile  :"+emobile);
20)       return "";
21)    }
22) }
23) class Test
24) {
25)    public static void main(String[] args)
26)    {
27)       Employee emp=new Employee();
28)       System.out.println(emp);
29)    }
30) }
```

Note: In Java, some predefined classes like String, StringBuffer, Exception, Thread, all wrapper classes, all Collection classes are not depending on Object class toString() method, they are having their own toString() method inorder to display their own data.

EX:

```
1)  class Test
2)  {
3)     public static void main(String[] args)
```

```
4)    {
5)        String str=new String("abc");
6)        System.out.println(str);
7)
8)        Exception e=new Exception("My Own Exception");
9)        System.out.println(e);
10)
11)       Thread t=new Thread();
12)       System.out.println(t);
13)
14)       Integer in=new Integer(10);
15)       System.out.println(in);
16)
17)       java.util.ArrayList al=new java.util.ArrayList();
18)       al.add("AAA");
19)       al.add("BBB");
20)       al.add("CCC");
21)       al.add("DDD");
22)       System.out.println(al);
23)   }
24) }
```

OUTPUT:

```
abc
java.lang.Exception: My Own Exception
Thread[Thread-0,5,main]
10
[AAA,BBB,CCC,DDD]
```

In JAVA, there are two types of Objects.

1) 1.Immutable Objects
2) 2.Mutable Objects

Q) What is the difference between *Immutable object* and *mutable object?*

OR

Q) What is the difference between *String* and *StringBuffer?*

Immutable Objects are Java objects; they will not allow modifications on their content. If we are trying to perform operations over immutable objects content then data is allowed for operations, but, the resultant data is not stored back in original object, the modified data will be stored by creating new Object.

EX: All String class objects are immutable objects.
All Wrapper class objects are immutable Objects.

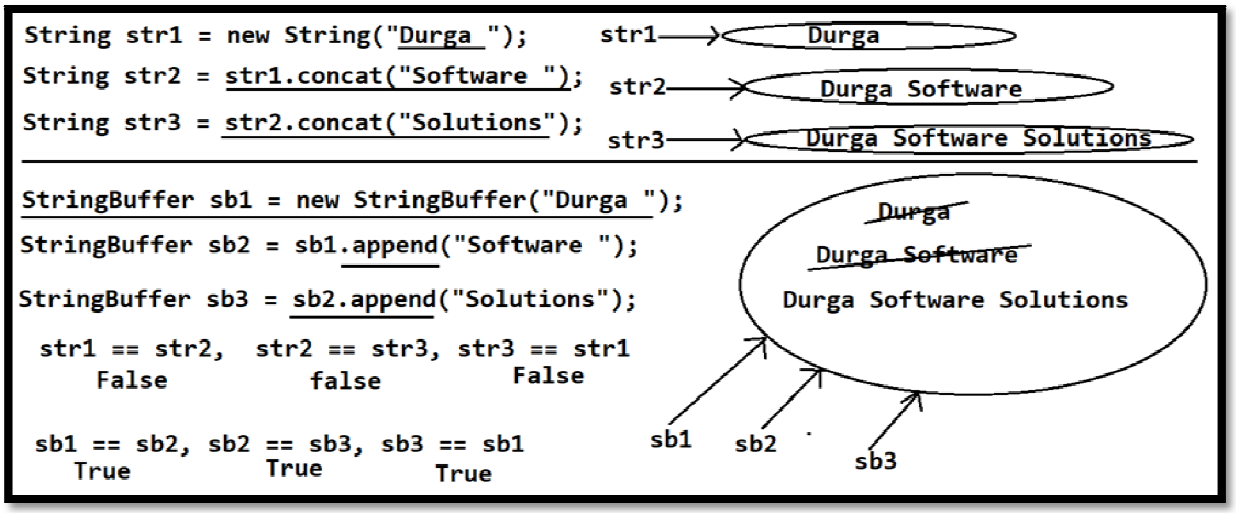
Mutable Objects are java objects, they will allow modifications on their content directly.
EX: Bydefault, all JAVA objects are mutable objects.
EX: StringBuffer

EX:

```
1)  class Test
2)  {
3)     public static void main(String[] args)
4)     {
5)        String str1=new String("Durga ");
6)        String str2=str1.concat("Software ");
7)        String str3=str2.concat("Solutions");
8)        System.out.println(str1);
9)        System.out.println(str2);
10)       System.out.println(str3);
11)       System.out.println();
12)       StringBuffer sb1=new StringBuffer("Durga ");
13)       StringBuffer sb2=sb1.append("Software ");
14)       StringBuffer sb3=sb2.append("Solutions");
15)       System.out.println(sb1);
16)       System.out.println(sb2);
17)       System.out.println(sb3);
18)    }
19) }
```

OUTPUT:
Durga
Durga Software
Durga Software Solutions

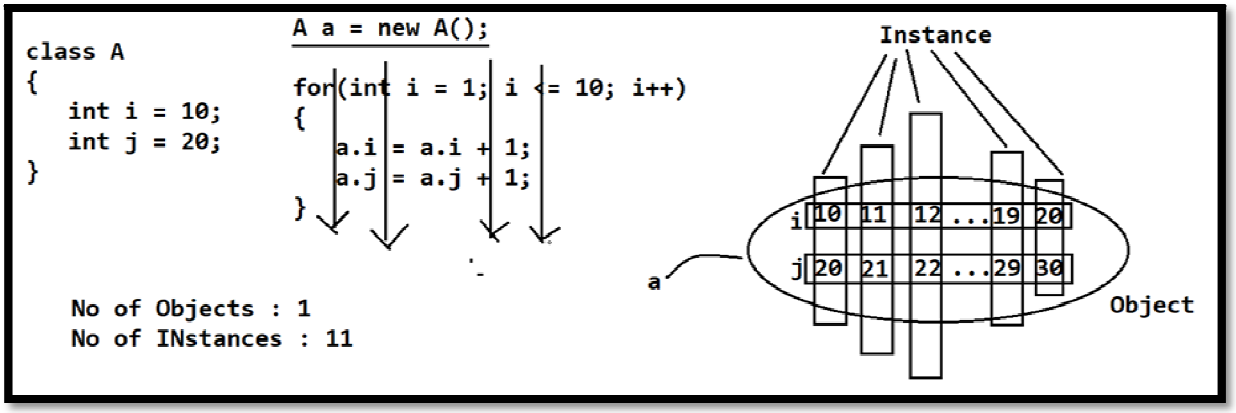
Durga Software Solutions
Durga Software Solutions
Durga Software Solutions



Q) What is the difference between Object and instance?

Object is a memory unit to store data.

Instance is a copy or layer of values existed in an object at a particular point of time.



Constructors:

1) **Constructor is a java feature; it can be used to create Object.**
2) **The role of the constructors in Object creation is to provide initial values inside the object.**
3) **In java applications, Constructors are executed exactly at the time of creating objects, not before creating objects and not after creating objects.**
4) **The main utilization of constructors is to provide initializations for class level variables mainly instance variable.**
5) **Constructors must have same name of the respective class.**
6) **Constructors are not having return types.**
7) **Constructors are not allowing the access modifiers like static, final,...**
8) **Constructors are able to allow the access modifiers like public, protected,<default> and private.**

9) Constructors are allowing 'throws' keyword in its syntax to bypass exceptions from present constructor to the caller.

Syntax: [Access_Modifier] Class_Name([Param_List])[throws Exception_List]
{

}

Note 1:

If we provide constructor name other than class name then compiler will rise an error like "Invalid Method declaration, return type required", because, compiler has treated the provided constructor as normal java method without the return type, but, for methods return is mandatory.

EX:

```
1)  class A
2)  {
3)     B()
4)     {
5)        System.out.println("A-Con");
6)     }
7)  }
8)  class Test
9)  {
10)    public static void main(String[] args)
11)    {
12)       A a=new A();
13)    }
14) }
```

Status: Compilation Error

Note 2:

If we provide return type to the constructors then Compiler will not rise any error and JVM will not rise any exception and JVM will not provide any output, because, the provided constructor is converted as normal java method. In this context, if we access the provided constructor as normal java method then it will be executed as like normal java method.

EX:

```
1)  class A
2)  {
3)     void A()
```

```
4)    {
5)       System.out.println("A-Con");
6)    }
7) }
8) class Test
9) {
10)   public static void main(String[] args)
11)   {
12)      A a=new A();
13)      a.A();
14)   }
15) }
```

Status: No Compialtion Error
OUTPUT: A-Con

Note 3:

If we provide the access modifiers like static, final,... to the constructors then Compiler will rise an error like "modifier xxx not allowed here".

EX:

```
1) class A
2) {
3)    static A()
4)    {
5)       System.out.println("A-Con");
6)    }
7) }
8) class Test
9) {
10)   public static void main(String[] args)
11)   {
12)      A a=new A();
13)
14)   }
15) }
```

Status: Compilation Error.

Note 4:

If we declare constructor as "private" then that constructor is available upto the respective class only, not available to out side of the respective class.
In this context, If we want to create object for the respective class then it is possible inside the same class only.

EX:

```
1)  class A
2)  {
3)     private A()
4)     {
5)        System.out.println("A-Con");
6)     }
7)  }
8)  class Test
9)  {
10)    public static void main(String[] args)
11)    {
12)       A a=new A();
13)    }
14) }
```

Status: Compilation Error

There are two types constructors in java

1) **Default Constructors**
2) **User Defined Constructors**

1) Default Constructors:

If we have not provided any constructor explicitly in java class then compiler will provide a 0-arg constructor automatically, here the compiler provided 0-arg constructor is called as "Default Constructor".

If we provide any constructor explicitly then compiler will not provide any default constructor.

D:\javaapps\ Test.java

```
public class Test {
}
```

On Command Prompt:

D:\javaapps>javac Test.java

D:\javaapps>javap Test

Compiled from Test.java

```
public class Test {
    public Test() { -----> Default constructor
    }
}
```

Note: Default Constructors are having the same scope of the respective class.

2) User Defined Constructors:

- These constructors are provided by the developers as per their application requirements.
- If we provide any constructor explicitly without parameters then that constructor is called as 0-arg constructor.
- If we provide any constructor with at least one parameter then that constructor is called as parameterized Constructor.

Note: By default, all the default constructors are 0-arg constructors, but, all 0-arg constructors are not default constructors, some 0-arg constructors are provided by the compiler are called as Default Constructors and some other 0-arg constructors are provided by the users they are called as User defined constructors.

EX:

```
1)  class Employee
2)  {
3)     String eid;
4)     String ename;
5)     float esal;
6)     String eaddr;
7)
8)     Employee()
9)     {
10)       eid="E-111";
11)       ename="Durga";
12)       esal=50000.0f;
13)       eaddr="Hyd";
14)    }
15)
16)    public void getEmpDetails()
17)    {
18)       System.out.println("Employee Details");
19)       System.out.println("----------------------");
20)       System.out.println("Employee Id      :"+eid);
21)       System.out.println("Employee Name    :"+ename);
22)       System.out.println("Employee Salary  :"+esal);
23)       System.out.println("Employee Address :"+eaddr);
24)    }
25) }
26) class Test
27) {
28)    public static void main(String[] args)
29)    {
```

```
30)       Employee emp=new Employee();
31)       emp.getEmpDetails();
32)    }
33) }
```

In the above program, if we create more than one object for the Employee class by executing 0-arg constructor then same data will be stored in all multiple objects of the Employee.

As per the requirement, if we want to create multiple Employee objects with different data then we have to provide data to the Objects explicitly, for this we have to use parameterized constructor.

EX:

```
1)  class Employee
2)  {
3)     String eid;
4)     String ename;
5)     float esal;
6)     String eaddr;
7)
8)     Employee(String emp_Id, String emp_Name, float emp_Sal, String emp_Addr)
9)     {
10)       eid=emp_Id;
11)       ename=emp_Name;
12)       esal=emp_Sal;
13)       eaddr=emp_Addr;
14)    }
15)
16)    public void getEmpDetails()
17)    {
18)       System.out.println("Employee Details");
19)       System.out.println("----------------------");
20)       System.out.println("Employee Id      :"+eid);
21)       System.out.println("Employee Name    :"+ename);
22)       System.out.println("Employee Salary  :"+esal);
23)       System.out.println("Employee Address :"+eaddr);
24)    }
25) }
26) class Test
27) {
28)    public static void main(String[] args)
29)    {
30)       Employee emp1=new Employee("E-111", "Durga", 50000.0f, "Hyd");
31)       emp1.getEmpDetails();
```

```
32)       System.out.println();
33)       Employee emp2=new Employee("E-222", "Anil", 60000.0f, "Hyd");
34)       emp2.getEmpDetails();
35)       System.out.println();
36)       Employee emp3=new Employee("E-333", "Rahul", 80000.0f, "Hyd");
37)       emp3.getEmpDetails();
38)    }
39) }
```

Constructor Overloading:

If we declare more than one constructor with the same name and with the different parameter list then it is called as "Constructor Overloading".

EX:

```
1)  class A
2)  {
3)     int i,j,k;
4)     A()
5)     {
6)     }
7)     A(int i1)
8)     {
9)        i=i1;
10)    }
11)    A(int i1, int j1)
12)    {
13)       i=i1;
14)       j=j1;
15)    }
16)    A(int i1, int j1, int k1)
17)    {
18)       i=i1;
19)       j=j1;
20)       k=k1;
21)    }
22)    void add()
23)    {
24)       System.out.println("Addition  :"+(i+j+k));
25)    }
26) }
27) class Test
28) {
29)    public static void main(String[] args)
30)    {
```

```
31)       A a1=new A();
32)       a1.add();
33)       A a2=new A(10);
34)       a2.add();
35)       A a3=new A(10,20);
36)       a3.add();
37)       A a4=new A(10,20,30);
38)       a4.add();
39)    }
40) }
```

Note: If we declare more than one method with the same name and with different parameter list then it is called as "Method Overloading"

Instance Context/Instance Flow of Execution:

In Java, for every class loading a separate context will be created called as Static Context and for every Object a separate context will be created called as Instance context.

In Java, instance context is represented in the form of the following elements.

1) Instance Variables
2) Instance Methods
3) Instance Blocks

1) Instance Variables:

* Instance Variable is a normal Java variable, whose values will be varied from one instance to another instance of an object.

* Instance Variable is a variable, which will be recognized and initialized just before executing the respective class constructor.

* In Java applications, instance variables must be declared at class level and non-static, instance variables never be declared as local variables and static variables.

* In Java applications, instance variables data will be stored in Object memory that is in "Heap Memory".

2) Instance Methods:

* Instance Method is a normal Java method, it is a set of instructions, it will represent an action of an entity.
* In Java applications, instance methods will be executed when we access that method. In Java applications, all the methods won't be executed without the method call.

EX:

```
1)  class A {
2)      int i=m1();
3)      A() {
4)       System.out.println("A-Con");
5)      }
6)      int m1() {
7)        System.out.println("M1-A");
8)        return 10;
9)      }
10) }
11) class Test {
12)    public static void main(String args[]) {
13)       A a = new A();
14)    }
15) }
```

OUTPUT:

M1-A
A-con

EX:

```
1)  class A {
2)      int j = m1();
3)      int m2() {
4)           System.out.println("m2-A");
5)           return 10;
6)      }
7)      A() {
8)              System.out.println("A-con");
9)      }
10)     int m1() {
11)                 System.out.println("m1-A");
12)                 return 20;
13)     }
14)     int i = m2();
```

```
15) }
16) class Test {
17)         public static void main(String args[]) {
18)                 A a = new A();
19)         }
20) }
```

OUTPUT: m1-A
m2-A
A-con

3) Instance Block:

- Instance Block is a set of instructions which will be recognized and executed just before executing the respective class constructors.
- Instance Block as are having the same power of constructors, it can be used as like constructors.

Syntax:

```
{
        ---instructions----
}
```

EX:

```
1)  class A {
2)      A() {
3)              System.out.println("A-CON");
4)      }
5)      {
6)          System.out.println("IB-A");
7)      }
8)  }
9)  class Test {
10)         public static void main(String args[]) {
11)             A a = new A();
12)         }
13) }
```

OUTPUT:
IB-A
A-CON

EX:

```
1)  class A {
```

```
2)      A() {
3)              System.out.println("A-CON");
4)      }
5)      {
6)              System.out.println("IB-A");
7)      }
8)      int m1() {
9)          System.out.println("m1-A");
10)         return 10;
11)     }
12)     int i = m1();
13) }
14) class Test {
15)     public static void main(String args[]) {
16)         A a = new A();
17)     }
18) }
```

OUTPUT: IB-A
m1-A
A-CON

EX:

```
1)  class A {
2)      int m1() {
3)          System.out.println("m1-A");
4)          return 10;
5)      }
6)      {
7)        System.out.println("IB-A");
8)      }
9)      int i=m2();
10)     A() {
11)         System.out.println("A-con");
12)     }
13)     int i=m1();
14)     {
15)         System.out.println("IB1-A");
16)     }
17)     int m2();
18)     {
19)       System.out.println("m2-A");
20)       return 20;
21)     }
22) }
```

```
23) class Test {
24)     public static void main(String args[]) {
25)         A a1=new A();
26)         A a2=new A();
27)     }
28) }
```

OUTPUT:

```
IB-A
m2-A
m1-A
IB1-A
A-con
IB-A
m2-A
m1-A
IB1-A
A-con
```

'this' Keyword:

'this' is a Java keyword, it can be used to represent current class object.

In Java applications, we are able to utilize 'this' keyword in the following 4 ways.

1) **To refer current class variable**
2) **To refer current class methods**
3) **To refer current class constructors**
4) **To refer current class object**

1) To Refer Current Class Variables:

If we want to refer current class variables by using 'this' keyword then we have to use the following syntax.

```
this.var_Name
```

NOTE: In Java applications, if we provide same set of variables at local and at class level and if we access that variables then JVM will give first priority for local variables, if local variables are not available then JVM will search for that variables at class level, even at class level also if that variables are not available then JVM will search at super class level. At all the above locations, if the specified variables are not available then compiler will rise an error.

NOTE: In Java applications, if we have same set of variables at local and at class level then to access class level variables over local variables we have to use 'this' keyword.

EX:

```
1)  class A {
2)        int i=10;
3)        int j=20;
4)        A(int i,int j) {
5)              System.out.println(i+" "+j);
6)              System.out.println(this.i+" "+this.j);
7)        }
8)  }
9)  class Test {
10)        public static void main(String args[]) {
11)              A a = new A(30,40);
12)        }
13) }
```

OUTPUT:
30 40
10 20

NOTE: In general, in enterprise applications, we are able to write no. of Java bean classes as per application requirement. In Java bean classes, we are able to provide variables and their setter methods and getter methods. In Java bean classes, in setter methods, we have to declare parameter variable with the same name of the respective class level variable, here we have to transfer data from local variable to class level variable, for this, we have to assign local variable to class level variable. In this context, to refer class level variable over local variable separately we have to use 'this' keyword. In getter methods, always, we have to return class level variables only, so that, it is not required to use 'this' keyword.

EX:

```
1)  class User {
2)        private String uname;
3)        private String upwd;
4)        public void setUname(String uname) {
5)              this.uname = uname;
6)        }
7)        public void setUpwd(String upwd) {
8)              this.upwd = upwd;
9)        }
10)       public getUname() {
11)          return uname;
12)       }
13)       public getUpwd() {
14)       }
15) }
```

```
16) class Test {
17)         public static void main(String[] args) {
18)                 User u = new User();
19)                 u.setUname("abc");
20)                 u.setUpwd("abc123");
21)                 System.out.println("User Login Details");
22)                 System.out.println("---------------------");
23)                 System.out.println("User Name   :"+u.getUname());
24)                 System.out.println("Password    :"+getEaddr());
25)         }
26) }
```

EX:

```
1)  class Employee
2)  {
3)     private String eid;
4)     private String ename;
5)     private float esal;
6)     private String eaddr;
7)
8)     public void setEid(String eid)
9)     {
10)       this.eid=eid;
11)    }
12)    public void setEname(String ename)
13)    {
14)       this.ename=ename;
15)    }
16)    public void setEsal(float esal)
17)    {
18)       this.esal=esal;
19)    }
20)    public void setEaddr(String eaddr)
21)    {
22)       this.eaddr=eaddr;
23)    }
24)
25)    public String getEid()
26)    {
27)       return eid;
28)    }
29)    public String getEname()
30)    {
31)       return ename;
32)    }
```

```
33)    public float getEsal()
34)    {
35)       return esal;
36)    }
37)    public String getEaddr()
38)    {
39)       return eaddr;
40)    }
41) }
42) class Test
43) {
44)    public static void main(String[] args)
45)    {
46)       Employee emp=new Employee();
47)       emp.setEid("E-111");
48)       emp.setEname("AAA");
49)       emp.setEsal(5000.0f);
50)       emp.setEaddr("Hyd");
51)
52)       System.out.println("Employee Details");
53)       System.out.println("--------------------");
54)       System.out.println("Employee Id     :"+emp.getEid());
55)       System.out.println("Employee Name   :"+emp.getEname());
56)       System.out.println("Employee Salary  :"+emp.getEsal());
57)       System.out.println("Employee Address :"+emp.getEaddr());
58)    }
59) }
```

2) To Refer Current Class Method:

If we want to refer current class method by using 'this' keyword then we have to use the following syntax.

this.method_Name([param_List]);

Note: To access current class methods, it is not required to use any reference variable and any keyword including 'this', directly we can access.

EX:

```
1)  class A {
2)     void m1() {
3)        System.out.println("m1-A");
4)     }
5)     void m2() {
6)         System.out.println("m2-A");
```

```
7)          m1();
8)          this.m1();
9)      }
10) }
11) class Test {
12)     public static void main(String args[]) {
13)          A a = new A();
14)          a.m2();
15)     }
16) }
```

OUTPUT:

```
m2-A
m1-A
m1-A
```

3) To Refer Current Class Constructors:

If we want to refer current class constructor by using 'this' keyword then we have to use the following syntax.

```
this([param_List]);
```

EX:

```
1)  class A {
2)      A() {
3)              this(10);
4)              System.out.println("A-0-arg-con");
5)      }
6)      A(int i) {
7)                  this(22.22f);
8)                  System.out.println("A-int-param-con");
9)      }
10)     A(float f) {
11)                 this(33.3333);
12)                 System.out.println("A-float-param-con");
13)     }
14)     A(double d) {
15)             System.out.println("A-double-param-con");
16)     }
17) }
18) class Test {
19)     public static void main(String args[]) {
20)             A a = new A();
21)     }
```

```
22) }
```

OUTPUT:

```
A-double-param-con
A-float-param-con
A-int-param-con
A-0-arg-con
```

NOTE: In the above program we have provided more than one constructor with the same name and with the different parameter list, this process is called as "Constructor Overloading".
In the above program, we have called all the current class constructors by using 'this' keyword in chain fashion, this process is called as "Constructor Chaining".

NOTE: If we want to access current class constructor by using 'this' keyword then the respective 'this' statement must be provided as first statement, if we have not provided 'this' statement as first statement then compiler will rise an error like 'call to this must be first statement in constructor'.

EX:

```
1)  class A {
2)      A() {
3)              System.out.println("A-0-arg-con");
4)              this(10);
5)      }
6)      A(int i) {
7)              System.out.println("A-int-param-con");
8)              this(22.22f);
9)      }
10)     A(float f) {
11)             System.out.println("A-float-param-con");
12)             this(33.3333);
13)     }
14)     A(double d) {
15)             System.out.println("A-double-param-con");
16)     }
17) }
18) class Test {
19)     public static void main(String args[]) {
20)             A a = new A();
21)     }
22) }
```

Status: Compilation Error

NOTE: If we want to refer current class constructor by using 'this' keyword then the respective 'this' statement must be provided in the current class another constructor only, not in normal Java methods. If we violate this condition then compiler will rise an error like 'call to this must be first statement in constructors'.

EX:

```
1)  class A
2)  {
3)     A()
4)     {
5)        System.out.println("A-0-arg-con");
6)     }
7)     A(int i)
8)     {
9)        System.out.println("A-int-param-con");
10)    }
11)    void m1()
12)    {
13)       this(10);
14)       System.out.println("m1-A");
15)    }
16) }
17) class Test
18) {
19)    public static void main(String args[])
20)    {
21)       A a=new A();
22)       a.m1();
23)    }
24) }
```

Status: Compilation Error.

Q) Is it possible to refer more than one current class constructors by using 'this' keyword from a single current class constructor?

No, It is not possible to refer more than one current class constructors by using 'this' keyword, because, in constructors, 'this' statement must be first statement while referring current class constructors. If we provide more than one time this(---) statement then only one this() statement is first statement among multiple this statements, in this case, compiler will rise an error like 'call to this must be first statement'.

EX:

```
1)  class A
2)  {
3)     A()
4)     {
5)        this(10);
6)        this(22.22f);
7)        System.out.println("A-0-arg-con");
8)     }
9)     A(int i)
10)    {
11)       System.out.println("A-int-param-con");
12)    }
13)    A(float f)
14)    {
15)       System.out.println("A-float-param-con");
16)    }
17) }
18) class Test
19) {
20)    public static void main(String args[])
21)    {
22)       A a=new A();
23)    }
24) }
```

Status: Compilation Error

4) To Return Current Class Object:

To return current class object by using 'this' keyword thn we have to use the followng syntax.

return this;

EX:

```
1)  class A {
2)        A getRef1() {
3)                          A a = new A();
4)                          return a;
5)        }
6)        A getRef2() {
7)                         return this;
8)        }
```

```
9)   }
10) class Test {
11)         public static void main(String args[]) {
12)                 A a = new A();//[a=abc123]
13)                 System.out.println(a);
14)                 System.out.println();
15)                 System.out.println(a.getRef1());//[abc123.getRef1()]
16)                 System.out.println(a.getRef1());//[abc123.getRef1()]
17)                 System.out.println(a.getRef1());//[abc123.getRef1()]
18)                 System.out.println();
19)                 System.out.println(a.getRef2());//[abc123.getRef2()]
20)                 System.out.println(a.getRef2());//[abc123.getRef2()]
21)                 System.out.println(a.getRef2());//[abc123.getRef2()]
22)     }
23) }
```

OUTPUT:

```
A@5e3a78ad

A@50c8d62f
A@3165d118
A@138297fe

A@5e3a78ad
A@5e3a78ad
A@5e3a78ad
```

In the above program, for every call of getRef1() method JVM will encounter "new" keyword, JVM will create new Object for class A every time and JVM will return new object reference value every time. This approach will increase no. of objects in Java application, it will not provide Objects reusability, it will provide object duplication, it is not suggestible in application development.

In the above program, for every call of getRef2() method JVM will encounter "return this" statement, JVM will not create new Object for class A every time, JVM will return the same reference value on which we have called getRef2() method. This approach will increase Objects reusability.

'static' keyword:

'static' is a Java keyword, it will improve sharability in Java applications.
In Java applications, static keyword will be utilized in the following four ways.

1) Static variables
2) Static methods
3) Static blocks
4) Static import

1) Static variables:

- Static variables are normal Java variables, which will be recognized and executed exactly at the time of loading the respective class byte code to the memory.

- Static variables are normal java variables, they will share their last modified values to the future objects and to the past objects of the respective class.

- In Java applications, static variables will be accessed either by using the respective class reference variable or by using the respective class name directly.

NOTE: To access static variables we can use the respective class reference variable which may or may not have reference value. To access static variables it is sufficient to take a reference variable with null value.

NOTE: If we access any non-static variable by using a reference variable with null value then JVM will rise an exception like "java.lang.NullPointerException". If we access static variable by using a reference variable contains null value then JVM will not rise any exception.
In Java applications, static variables must be declared as class level variables only, they never be declared as local variables.
In Java applications, static variable values will be stored in method area, not in Stack memory and not in Heap Memory.
In Java applications, to access current class static variables we can use "this" keyword.
EX:

```
1)  class A {
2)      static int i=10;
3)      int j = 10;
4)      void m1() {
5)              //static int k=30;--->error
6)              System.out.println("m1-A");
7)              System.out.println(this.i);
8)      }
9)  }
10) class Test {
11)     public static void main(String args[]) {
12)             A a = new A();
13)             System.out.println(a.i);
14)             System.out.println(A.i);
15)             a.m1();
```

```
16)                 A a1 = null;
17)                 //System.out.println(a1.j);--->NullPointerException
18)                 System.out.println(a1.i);
19)       }
20) }
```

OUTPUT:

```
10
10
m1-A
10
10
```

EX:

```
1)  class A {
2)      static int i=10;
3)      int j=10;
4)  }
5)  class Test {
6)       public static void main(String args[]) {
7)          A a1 = new A();
8)          System.out.println(a1.i+"   "+a1.j);
9)          a1.i=a1.i+1;
10)         a1.j=a1.j+1;
11)         System.out.println(a1.i+"   "+a1.j);
12)         A a2 = new A();
13)         System.out.println(a2.i+"   "+a2.j);
14)         a2.i=a2.i+1;
15)         a2.j=a2.j+1;
16)         System.out.println(a1.i+"   "+a1.j);
17)         System.out.println(a2.i+"   "+a2.j);
18)         A a3 = new A();
19)         System.out.println(a3.i+"   "+a3.j);
20)         a3.i=a3.i+1;
21)         a3.j = a3.j+1;
22)         System.out.println(a1.i+"   "+a1.j);
23)         System.out.println(a2.i+"   "+a2.j);
24)         System.out.println(a3.i+"   "+a3.j);
25)     }
26) }
```

OUTPUT:

```
10   10
11   11
11   10
```

12 11
12 11
12 11
12 10
13 11
13 11
13 11

NOTE: In Java applications, Instance variable is specific to each and every object that is a separate copy of instance variables will be maintained by each and every object but static variable is common to every object that is the same copy of static variable will be maintained by all the objects of the respective class.

Note: In Java, for a particular, class Byte-Code will be loaded only one time but we can create any number of objects.

2) Static Methods:

- Static method is a normal java method, it will be recognized and executed the time when we access that method.
- Static methods will be accessed either by using reference variable or by using the respective class name directly.

Note: In the case of accessing static methods by using reference variables, reference variable may or may not have object reference value, it is possible to access static methods with the reference variables having 'null' value. If we access non-static method by using a reference variable contains null value then JVM will rise an exception like "java.lang.NullPointerException".
Static methods will allow only static members of the current class, static methods will not allow non-static members of the current class directly.

Note: If we want to access non-static members of the current class in static methods then we have to create an object for the current class and we have to use the generated reference variable.

Static methods are not allowing 'this' keyword in its body but to access current class static methods we are able to use 'this' keyword.

EX:

```
1)  class A {
2)      int I = 10;
3)      static int j = 20;
4)      static void m1() {
5)          System.out.println("m1-A");
6)          System.out.println(j);
```